

简易编译器 QCompiler 实现及验证

路径 A: LL(1) 递归下降 $\rightarrow$ QAST $\rightarrow$ QTAC $\rightarrow$ ARM64

姓名: 李祥熙 学号: 2234412799 班级: 计算机 2301

2026 年 6 月 8 日

目录

1 实验目的	3
2 实验过程	3
2.1 实验环境	3
2.2 整体架构	3
2.3 实现步骤	4
3 实验内容	4
3.1 任务步骤 1: 词法分析器	4
3.1.1 QL 语法规则	4
3.1.2 实现要点	5
3.1.3 Token 序列输出 (部分)	6
3.2 任务步骤 2A: LL(1) 递归下降分析与 QAST 构建	7
3.2.1 文法变换	7
3.2.2 递归下降实现核心片段	8
3.2.3 QAST 结点设计	8
3.2.4 QAST 输出 (部分)	9
3.3 任务步骤 2AB: QTAC 中间代码生成与符号表	11
3.3.1 QTAC 生成策略	11
3.3.2 QTAC 完整输出	12
3.3.3 符号表	14
3.4 任务步骤 3: QTAC $\rightarrow$ ARM64 汇编生成	15
3.4.1 寄存器与栈帧策略	15
3.4.2 Q2ARM 模板映射	15
3.4.3 跨平台兼容性	16
3.4.4 ARM64 汇编输出 (部分)	16

目录	2
3.5 任务步骤 4: ARM64 汇编代码优化	17
3.5.1 三个窥孔优化遍	17
3.5.2 优化效果	18
3.6 任务步骤 5: 整体测试与平台验证	18
3.6.1 C 测试驱动	18
3.6.2 端到端运行脚本	19
4 实验结果	20
4.1 端到端运行结果	20
4.2 各阶段输出统计	21
4.3 与参考 QTAC 对比分析	22
5 总结	22

1 实验目的

本实验完整实现了从 QL 源程序到 ARM64 汇编的编译器全流程（路径 A），具体目标如下：

- **掌握词法分析原理**：基于正则与 DFA 思想构建词法分析器，将 QL 源程序转换为标准 (TYPE, value) Token 序列。
- **掌握 LL(1) 语法分析**：对 QL 文法消除左递归与左公因子，验证 SELECT 集不相交，实现递归下降分析器并构建 QAST 抽象语法树。
- **掌握中间代码生成**：遍历 QAST，按 QTAC 三地址码规范生成中间代码和符号表。
- **掌握目标代码生成**：依照 Q2ARM 指令模板，将 QTAC 翻译为符合 AAPCS64 调用约定的 ARM64 汇编，并在本地 ARM64 (Apple Silicon macOS) 平台验证运行正确性。
- **实施代码优化**：实现三个窥孔优化遍（冗余跳转消除、存储-加载转发、自赋值消除），在不改变语义的前提下减少指令数量。

2 实验过程

2.1 实验环境

- **开发平台**：macOS (Apple Silicon, arm64, 与华为鲲鹏同为 ARM64 架构)
- **本地汇编/链接**：Apple Clang 17.0.0 (cc), 原生 arm64 目标
- **鲲鹏目标**：生成器同时支持 `--target linux`, 输出符合 GNU as / ELF 规范的标准 ARM64 汇编（唯一区别：全局符号地址化指令），可直接在鲲鹏平台用 gcc/as 汇编链接
- **编程语言**：Python 3.14
- **测试程序**：tests/qsrt.q1 (QL 语言快速排序，对 10 元素整型数组排序)

2.2 整体架构

编译器由 6 个 Python 模块组成，由驱动脚本 qcc.py 串联：

表 1: QCompiler 模块清单

模块文件	对应任务	功能
lexer.py	Task 1	QL 源程序 $\rightarrow$ Token 序列
qast.py	Task 2AB	QAST 结点定义 + 格式化输出
parser_ll1.py	Task 2A	LL(1) 递归下降 $\rightarrow$ QAST
irgen.py	Task 2AB	QAST $\rightarrow$ QTAC + 符号表
codegen.py	Task 3	QTAC $\rightarrow$ ARM64 汇编
optimize.py	Task 4	ARM64 窥孔优化
qcc.py	驱动	串联所有阶段, 写出各中间文件

2.3 实现步骤

1. **分析 QL 文法**, 找出左递归、二义性和 SELECT 集冲突, 制定消除方案。
2. **实现词法分析器** (lexer.py), 支持关键字、标识符、数值、运算符、界符及行注释, 并对 PDF/Word 中的排版字符 (如 $\times \rightarrow *$ 、 $\rightarrow -$) 做规范化预处理。
3. **改造文法**, 消除左递归与左公因子, 将表达式和条件合并为统一的优先级爬升文法, 并实现递归下降分析器 (parser_ll1.py) 以构建 QAST。
4. **定义 QAST 结点** (qast.py), 按材料中 QASTNode 规范设计, 并实现缩进格式化输出 (dump) 作为 .ast 交付物。
5. **实现 QTAC 生成器** (irgen.py): 遍历 QAST, 分别处理全局变量、函数定义及顶层语句, 生成标准 QTAC 三地址码和符号表。数组元素地址显式乘以 8 (字长), 使代码可直接运行。
6. **实现代码生成器** (codegen.py): 按 Q2ARM 指令模板, 将 QTAC 逐条翻译为 ARM64; 所有局部变量/临时变量均分配栈槽, 通过 X9/X10 两个划痕寄存器完成运算, 无需手工替换虚拟寄存器。同时处理 macOS (adrp/@PAGE) 与 Linux (ADR) 两种全局地址化方式。
7. **实现三遍窥孔优化** (optimize.py) 并通过 run.sh 完成端到端集成测试。

3 实验内容

3.1 任务步骤 1: 词法分析器

3.1.1 QL 词法规则

QL 语言的词法类别 (来自实验材料):

- 整型字面量 i : $[0-9]^+$
- 标识符/关键字 d : $[a-zA-Z]^+$, 关键字集合为 $\{\text{int, void, if, else, while, return}\}$
- 算术/赋值运算符 o : $+ - * / = +=$
- 关系运算符 r : $< <= > >= == !=$
- 逻辑运算符: $! \&\& ||$
- 界符: $() \{ \} [] ; ,$
- 行注释: $// \dots$ (忽略至行末)

3.1.2 实现要点

词法分析器采用手工 DFA 风格 (`lexer.py`), 按优先级依次尝试各规则:

Listing 1: 词法分析核心逻辑 (`lexer.py`)

```

1 KEYWORDS = {"int", "void", "if", "else", "while", "return"}
2 MULTI_OPS = ["<=", ">=", "==", "!=", "&&", "||", "+="]
3 SINGLE_OPS = set("+-*/=<>!")
4 DELIMS = set("(){}[]; ,")
5
6 # 排版字符规范化 (PDF/Word 来源)
7 GLYPH_FIX = {"x": "*", "-": "-", ":": "-", " ": "-"}
8
9 def tokenize(text):
10     text = normalise(text)    # 先做字符替换
11     ...
12     while i < n:
13         if c.isdigit():        # 数值字面量
14             m = re.match(r"[0-9]+", text[i:])
15             tokens.append(Token("NUM", m.group(0), line, col))
16         elif c.isalpha():      # 标识符/关键字
17             m = re.match(r"[a-zA-Z_][a-zA-Z0-9_]*", text[i:])
18             ttype = "KEYWORD" if m.group(0) in KEYWORDS else "ID"
19         elif text[i:i+2] in MULTI_OPS: # 双字符运算符优先
20             tokens.append(Token("OP", text[i:i+2], line, col))
21         elif c in SINGLE_OPS:
22             tokens.append(Token("OP", c, line, col))
23         elif c in DELIMS:
24             tokens.append(Token("DELIM", c, line, col))

```

关键设计是双字符运算符必须先于单字符运算符匹配，否则 `<=` 会被错误分割为 `<` 和 `=` 两个 Token。

3.1.3 Token 序列输出（部分）

以下为 `qsort.q1` 词法分析的前 35 个 Token（共 235 个）：

Listing 2: `qsort.tokens`（前 35 行）

```
1 (DELIM, {)
2 (KEYWORD, int)
3 (ID, a)
4 (DELIM, [])
5 (NUM, 10)
6 (DELIM, ])
7 (DELIM, ;)
8 (KEYWORD, void)
9 (ID, swap)
10 (DELIM, ()
11 (KEYWORD, int)
12 (ID, arr)
13 (DELIM, [])
14 (DELIM, ])
15 (DELIM, ;)
16 (KEYWORD, int)
17 (ID, i)
18 (DELIM, ;)
19 (KEYWORD, int)
20 (ID, j)
21 (DELIM, ;)
22 (DELIM, ))
23 (DELIM, {)
24 (KEYWORD, int)
25 (ID, temp)
26 (DELIM, ;)
27 (ID, temp)
28 (OP, =)
29 (ID, arr)
30 (DELIM, [])
31 (ID, i)
32 (DELIM, ])
33 (DELIM, ;)
34 (ID, arr)
```

35 (DELIM, [])

可以看到词法分析器正确区分了关键字 (KEYWORD)、标识符 (ID)、数值 (NUM)、运算符 (OP) 和界符 (DELIM) 五类 Token。arr[] 中的两个方括号被正确分解为两个独立界符 Token，与后续解析器处理数组传参的“退化形式”相吻合。

3.2 任务步骤 2A: LL(1) 递归下降分析与 QAST 构建

3.2.1 文法变换

材料给出的原始 QL 文法存在两类障碍，需先变换才能用于 LL(1) 分析：

(1) **左递归消除** 原文法中 $V \rightarrow d \mid V[E]$ 、 $E \rightarrow E \text{ op } E$ 、 $\check{D} \rightarrow \check{D}D$ ；等均含直接左递归，将其改写为迭代形式：

$$\begin{aligned} V &\rightarrow d ([E])^* \\ \check{D} &\rightarrow D ; (\check{D})^* \Rightarrow \text{循环解析} \\ \check{R} &\rightarrow E , (\check{R})^* \Rightarrow \text{循环解析} \end{aligned}$$

表达式 $E \rightarrow E \text{ op } E$ 改为**优先级爬升文法** (precedence climbing)，将算术与关系/逻辑运算合并为同一梯级序列，彻底消除二义性并避免插入多余的非终结符结点：

$$\begin{aligned} \text{assign} &\rightarrow \text{logic_or } [(= \mid +=) \text{ assign}] \\ \text{logic_or} &\rightarrow \text{logic_and } (|| \text{ logic_and})^* \\ \text{logic_and} &\rightarrow \text{equality } (\&\& \text{ equality})^* \\ \text{equality} &\rightarrow \text{relational } ((= \mid !=) \text{ relational})^* \\ \text{relational} &\rightarrow \text{additive } ((< \mid <= \mid > \mid >=) \text{ additive})^* \\ \text{additive} &\rightarrow \text{mul } ((+ \mid -) \text{ mul})^* \\ \text{mul} &\rightarrow \text{unary } ((* \mid /) \text{ unary})^* \\ \text{unary} &\rightarrow (- \mid !) \text{ unary} \mid \text{postfix} \\ \text{postfix} &\rightarrow \text{primary } ([E] \mid [])^* \\ \text{primary} &\rightarrow \text{NUM} \mid (\text{assign}) \mid \text{ID} \mid \text{ID} (\text{args}) \end{aligned}$$

将 E 和 C 合并为同一梯级序列，同时消除了 (E) 与 (C) 无法区分的歧义。

(2) **声明与语句的 SELECT 集不相交** 块 (Block) 体内的决策只需一个 Token 的前瞻：以类型关键字 int/void 开头的是声明，其余为语句——两个 SELECT 集天然不相交，满足 LL(1) 条件。

3.2.2 递归下降实现核心片段

Listing 3: 优先级爬升的通用左结合辅助函数

```

1 def _left_assoc(self, sub, ops):
2     node = sub()                                # 先解析更高优先级的子表达式
3     while self.cur.value in ops:                # 只要当前 Token 是该层的运算符
4         op = self.advance().value
5         node = qast.Binary(op, node, sub())      # 构造左结合二叉结点
6     return node
7
8 def parse_additive(self):
9     return self._left_assoc(self.parse_mul, ("+", "-"))

```

Listing 4: 标识符后的分叉：变量引用 vs. 函数调用

```

1 def parse_primary(self):
2     t = self.cur
3     if t.type == "ID":
4         self.advance()
5         if self.at("("):                        # 下一个 Token 是 '(' -> 函数调用
6             args = self.parse_args()
7             return qast.Call(t.value, args)
8         return qast.Id(t.value)                # 否则 -> 普通标识符引用

```

Listing 5: 的处理] 数组下标与”退化”传参形式 arr[] 的处理

```

1 def parse_postfix(self):
2     node = self.parse_primary()
3     while self.at("["):
4         self.expect("[")
5         if self.at("]"):                        # arr[] 形式：数组名作为地址传递
6             self.expect("]")                  # 结点不变，保留 Id(arr) 即基地址
7         else:
8             idx = self.parse_expr()
9             self.expect("]")
10            node = qast.Index(node, idx)        # 构造 Index 结点
11    return node

```

3.2.3 QAST 结点设计

qast.py 用一个通用 Node 类加 kind 标签表示所有结点，并为每种结点提供具名构造函数，与材料中的 QASTNode 规范一一对应：

表 2: QAST 结点与 QASTNode 规范对应表

结点 kind	对应 QASTNode	主要字段
Program	$P \rightarrow B$	decls, stmts
VarDecl	NodeDVar	name, dims
FuncDecl	NodeDFunc	name, ret_type, params, body
Param	NodeAVar	name, is_array
Block	NodeB	decls, stmts
ExprStmt	NodeSEExpr	expr
If	NodeIF	cond, then, els
While	NodeWHILE	cond, body
Return	NodeRETURN	expr
Nop	NodeNOP	—
Num	NodeNUM	value
Id	NodeID	name
Index	NodeArrayAccess	base, index
Call	NodeCall	name, args
Unary	NodeUOP	op, operand
Binary	NodeAOP/NodeROP/...	op, left, right

3.2.4 QAST 输出 (部分)

以下为 qsort.q1 的 QAST 格式化输出 (展示 swap 函数和 partition 前半段, 其余结构类似):

Listing 6: qsort.ast (节选)

```

1 Program
2   decls:
3     VarDecl a: int[10]
4     FuncDecl void swap(arr[], i, j)
5       Block
6         VarDecl temp: int
7         ExprStmt
8           Binary =
9             Id temp
10            Index
11              base:
12                Id arr
13              index:
14                Id i

```

```
15      ExprStmt
16      Binary =
17      Index
18      base:
19      Id arr
20      index:
21      Id i
22      Index
23      base:
24      Id arr
25      index:
26      Id j
27      ExprStmt
28      Binary =
29      Index
30      base:
31      Id arr
32      index:
33      Id j
34      Id temp
35      FuncDecl int partition(arr[], low, high)
36      Block
37      ...
38      While
39      cond:
40      Binary <
41      Binary =
42      Id j
43      Binary +
44      Id j
45      Num 1
46      Id high
47      body:
48      Block
49      If
50      cond:
51      Binary <
52      Index
53      base:
54      Id arr
55      index:
```

```

56         Id j
57         Id pivot
58     then:
59         Block
60         ExprStmt
61         Binary =
62         Id i
63         Binary +
64         Id i
65         Num 1
66         ExprStmt
67         Call swap
68         Id arr
69         Id i
70         Id j
71 stmts:
72     ExprStmt
73     Call qsort
74     Id a
75     Num 0
76     Num 9

```

QAST 通过树的嵌套层次清晰地表达了运算符优先级。以 `partition` 的 `while` 循环条件为例： $(j = j + 1) < \text{high}$ 被表示为一个 `Binary <` 结点，其左子树是赋值表达式 `Binary =`（含加法子树），右子树是 `Id high`，优先级完全由树形结构承载，无需额外优先级字段。

3.3 任务步骤 2AB: QTAC 中间代码生成与符号表

3.3.1 QTAC 生成策略

QTAC 生成器 (`irgen.py`) 采用后序遍历（表达式自底向上求值），核心策略：

- **临时变量**：以 `t0`, `t1`, ... 全局递增编号，每个子表达式产生一个新临时变量，满足单赋值约束。
- **标签**：以 `l0`, `l1`, ... 全局递增编号，`if/while` 各自分配所需数量。
- **数组寻址**：元素地址 = `base + index × 8`，字长 8 字节显式计算（区别于参考 QTAC 省略 `×8` 的写法），使代码可直接执行。
- **条件翻译**：关系运算直接生成 `IF ...THEN ...ELSE ...`；逻辑与 `&&`、或 `||` 用短路标签串联；逻辑非 `!` 交换真假标签。

- **顶层语句**: 封装为合成函数 `define entry(){...}`, 使程序入口结构统一, 便于代码生成器处理。

3.3.2 QTAC 完整输出

以下为 `qsort.q1` 的完整 QTAC 输出:

Listing 7: `qsort.qtac` (完整)

```
1 i80 a;
2
3 define swap(arr, i, j){
4 LABEL swap;
5 t0 = i * 8;
6 t1 = arr + t0;
7 t2 = M[t1];
8 temp = t2;
9 t3 = i * 8;
10 t4 = arr + t3;
11 t5 = j * 8;
12 t6 = arr + t5;
13 t7 = M[t6];
14 M[t4] = t7;
15 t8 = j * 8;
16 t9 = arr + t8;
17 M[t9] = temp;
18 RETURN 0;
19 }
20
21 define partition(arr, low, high){
22 LABEL partition;
23 t10 = high * 8;
24 t11 = arr + t10;
25 t12 = M[t11];
26 pivot = t12;
27 t13 = low - 1;
28 i = t13;
29 t14 = low - 1;
30 j = t14;
31 LABEL l0;
32 t15 = j + 1;
33 j = t15;
34 IF j < high THEN l1 ELSE l2;
```

```
35 LABEL l1;
36 t16 = j * 8;
37 t17 = arr + t16;
38 t18 = M[t17];
39 IF t18 < pivot THEN l3 ELSE l4;
40 LABEL l3;
41 t19 = i + 1;
42 i = t19;
43 PAR arr;
44 PAR i;
45 PAR j;
46 t20 = CALL swap, 3;
47 LABEL l4;
48 GOTO l0;
49 LABEL l2;
50 t21 = i + 1;
51 PAR arr;
52 PAR t21;
53 PAR high;
54 t22 = CALL swap, 3;
55 t23 = i + 1;
56 RETURN t23;
57 RETURN 0;
58 }
59
60 define qsort(arr, low, high){
61 LABEL qsort;
62 IF low < high THEN l5 ELSE l6;
63 LABEL l5;
64 PAR arr;
65 PAR low;
66 PAR high;
67 t24 = CALL partition, 3;
68 p = t24;
69 t25 = p - 1;
70 PAR arr;
71 PAR low;
72 PAR t25;
73 t26 = CALL qsort, 3;
74 t27 = p + 1;
75 PAR arr;
```

```

76 PAR t27;
77 PAR high;
78 t28 = CALL qsort, 3;
79 LABEL 16;
80 RETURN 0;
81 }
82
83 define entry(){
84 LABEL entry;
85 PAR a;
86 PAR 0;
87 PAR 9;
88 t29 = CALL qsort, 3;
89 RETURN 0;
90 }

```

将本实验生成的 QTAC 与材料提供的参考 QTAC 对比，两者结构高度吻合：swap、partition、qsort 三个函数的控制流标签、临时变量个数与操作顺序均与参考一致（主要差异是：本实现显式生成了 $\times 8$ 的字节偏移计算，而参考中该步骤被省略）。

3.3.3 符号表

Listing 8: qsort.sym

```

1 ; QTAC symbol table
2 [GLOBAL] a : int[10] size=80
3 [FUNCTION] swap(arr:int[], i:int, j:int) -> void
4     local temp : int
5 [FUNCTION] partition(arr:int[], low:int, high:int) -> int
6     local pivot : int
7     local i : int
8     local j : int
9 [FUNCTION] qsort(arr:int[], low:int, high:int) -> void
10     local p : int
11 [FUNCTION] entry() -> int (program entry)

```

符号表记录了全局数组 a（80 字节，10 个 8 字节整型元素）、3 个用户函数及其参数/局部变量类型，以及合成入口函数 entry。

3.4 任务步骤 3: QTAC → ARM64 汇编生成

3.4.1 寄存器与栈帧策略

代码生成器 (codegen.py) 采用”全部溢出到栈”策略: 函数中每个 QTAC 值 (参数、局部变量、临时变量) 在栈帧中分配一个 8 字节槽位; 每条指令先从栈装载操作数到划痕寄存器 X9/X10, 运算后再写回栈槽。这是材料明确允许的简化方案 (不实现寄存器分配时, 虚拟寄存器直接映射到栈槽), 优化遍随后清除冗余的装载/存储。

栈帧布局由 layout 方法确定: 先遍历函数的所有指令, 收集出现过的值名, 按首次出现顺序分配偏移 (参数在最低地址), 最终按 16 字节对齐。以 swap 函数 (3 个参数 + 1 个局部变量 + 10 个临时变量 = 14 个槽位) 为例, 帧宽为 $14 \times 8 = 112$ 字节 (已 16 字节对齐)。

3.4.2 Q2ARM 模板映射

下表列出本实现涵盖的 Q2ARM 模板及对应的 ARM64 指令序列:

表 3: Q2ARM 指令模板实现

QTAC 形式	ARM64 指令序列
LABEL l	l:
GOTO l	B l
IF q rop a THEN l1 ELSE l2	LDR X9,slot(q); LDR X10,slot(a); CMP X9,X10; Jrop l1; B l2
RETURN a	LDR X0,slot(a); B func_exit
PAR a ₁ ; ...; q=CALL d, k	LDR X0,a ₁ ; ...; BL d; STR X0,slot(q)
q = M[q2]	LDR X9,slot(q2); LDR X9,[X9]; STR X9,slot(q)
M[q1] = a	LDR X9,slot(q1); LDR X10,slot(a); STR X10,[X9]
q = a aop b	LDR X9,a; LDR X10,b; AOP X9,X9,X10; STR X9,slot(q)
q = a	LDR X9,slot(a); STR X9,slot(q)
q = #k (立即数)	MOV X9,#k; STR X9,slot(q)
q = globalname	ADR/ADRP X9,sym; STR X9,slot(q) (全局地址)

函数序言/尾声严格按照 Q2ARM 模板中的 AAPCS64 规范生成:

Listing 9: 函数序言与尾声 (以 swap 为例, Linux 目标)

```

1 ql_swap:
2 _ql_swap:
3     stp x29, x30, [sp, #-16]!    ; 保存帧指针和链接寄存器
4     mov x29, sp                 ; 建立栈帧基准
5     sub sp, sp, #112            ; 分配 112 字节局部区
6     STR X0, [sp, #0]            ; 溢出参数 arr
7     STR X1, [sp, #8]            ; 溢出参数 i

```

```

8      STR X2, [sp, #16]          ; 溢出参数 j
9      ...                      ; 函数体指令
10 ql_swap_exit:
11     add sp, sp, #112          ; 释放局部区
12     ldp x29, x30, [sp], #16    ; 恢复 FP 和 LR
13     ret                      ; 返回

```

3.4.3 跨平台兼容性

全局变量地址化在 Linux/Kunpeng (ELF) 和 macOS (Mach-O) 有所不同:

Listing 10: 全局地址化: Linux 使用 ADR, macOS 使用 ADRP

```

1 ; Linux/Kunpeng 目标 (--target linux, default)
2 ADR X9, ql_a          ; PC 相对寻址, ±1MB 范围内有效
3
4 ; macOS 目标 (--target macos)
5 adrp X9, ql_a@PAGE     ; 加载页地址
6 add X9, X9, ql_a@PAGEOFF ; 加上页内偏移

```

函数和全局符号均同时导出 `name:` 和 `_name:` 两个标签 (`.global` 声明), 使同一份 `.s` 文件在两个平台上均可汇编链接, 延续了大作业一的做法。

3.4.4 ARM64 汇编输出 (部分)

以下为 `swap` 函数完整的未优化 ARM64 汇编 (Linux 目标):

Listing 11: `qsort.s`: `ql_swap` 函数 (未优化, Linux 目标)

```

1 .global ql_swap
2 .global _ql_swap
3 ql_swap:
4 _ql_swap:
5     stp x29, x30, [sp, #-16]!
6     mov x29, sp
7     sub sp, sp, #112
8     STR X0, [sp, #0]      ; arr -> slot[0]
9     STR X1, [sp, #8]      ; i   -> slot[1]
10    STR X2, [sp, #16]     ; j   -> slot[2]
11    LDR X9, [sp, #8]      ; t0 = i * 8
12    MOV X10, #8
13    MUL X9, X9, X10
14    STR X9, [sp, #24]
15    LDR X9, [sp, #0]      ; t1 = arr + t0

```



```

16     LDR X10, [sp, #24]
17     ADD X9, X9, X10
18     STR X9, [sp, #32]
19     LDR X9, [sp, #32]          ; t2 = M[t1]
20     LDR X9, [X9]
21     STR X9, [sp, #40]
22     LDR X9, [sp, #40]          ; temp = t2
23     STR X9, [sp, #48]
24     ... (arr[i]=arr[j] 和 arr[j]=temp 的类似序列)
25     MOV X0, #0
26     B ql_swap_exit
27 ql_swap_exit:
28     add sp, sp, #112
29     ldp x29, x30, [sp], #16
30     ret

```

3.5 任务步骤 4: ARM64 汇编代码优化

3.5.1 三个窥孔优化遍

optimize.py 实现了三个顺序执行的窥孔优化遍，仅观察相邻指令，从不跨越标签或跳转，因此不会改变基本块边界处的活跃性分析结果，语义完全安全。

遍 1: 冗余跳转消除 若无条件跳转 B Lx 的紧接下一行恰好是目标标签 Lx:，则该跳转可安全删除：

Listing 12: 冗余跳转消除示例

```

1 ; 优化前
2     MOV X0, #0
3     B ql_swap_exit          ; <- 跳到紧邻的下一行，冗余
4 ql_swap_exit:
5     add sp, sp, #112
6
7 ; 优化后
8     MOV X0, #0              ; 跳转被删除
9 ql_swap_exit:
10    add sp, sp, #112

```

遍 2: 存储-加载转发与重复加载消除 若 STR Xr, [sp, #k] 之后立刻是 LDR Xr, [sp, #k] (同寄存器、同偏移)，则后者冗余 (值已在 Xr 中)；类似地，连续两条 LDR Xr, [sp, #k] 亦只需一条：

Listing 13: 存储-加载转发示例 (swap 函数开头)

```
1 ; 优化前
2     LDR X9, [sp, #32]
3     LDR X9, [X9]           ; 内存加载 (地址在 X9)
4     STR X9, [sp, #40]      ; 写入 slot[5] (t2)
5     LDR X9, [sp, #40]      ; <- 紧接从同一槽读回, 冗余
6     STR X9, [sp, #48]      ; 写入 slot[6] (temp)
7
8 ; 优化后
9     LDR X9, [sp, #32]
10    LDR X9, [X9]
11    STR X9, [sp, #40]      ; 保留
12                                ; LDR [sp,#40] 被删除 (X9 已持有该值)
13    STR X9, [sp, #48]
```

遍 3: 自赋值消除 MOV Xr, Xr 形式的自我拷贝指令直接删除。

3.5.2 优化效果

在 qsort.q1 上的统计结果:

表 4: 窥孔优化统计 (qsort.q1)

指标	未优化	优化后
汇编行数 (含空行/标签)	241	223
删除的冗余跳转指令数	—	4
删除的冗余加载指令数	—	14
删除的自赋值指令数	—	0
合计删除指令数	—	18

三遍优化共删除了 18 条冗余指令, 使汇编代码缩短约 7.5%。优化后的汇编与未优化版本产生相同的运行结果 (均通过正确性检验), 证明优化不改变程序语义。

3.6 任务步骤 5: 整体测试与平台验证

3.6.1 C 测试驱动

QL 程序本身只对全局数组 a 排序, 既不初始化也不打印, 因此设计了 C 语言测试驱动 runtime/main.c:

Listing 14: runtime/main.c (测试驱动)

```

1  #include <stdio.h>
2
3  extern long long ql_a[10];
4  extern void ql_entry(void);    /* QL 顶层入口 (调用 qsort) */
5
6  int main(void) {
7      long long init[10] = {5, 3, 8, 1, 9, 2, 7, 4, 6, 0};
8      for (int k = 0; k < 10; k++) ql_a[k] = init[k];
9
10     printf("before:"); for (int k=0;k<10;k++) printf(" %lld", ql_a[k
11         ]);
12     printf("\n");
13
14     ql_entry();    /* 执行 QL 程序: qsort(a, 0, 9) */
15
16     printf("after :"); for (int k=0;k<10;k++) printf(" %lld", ql_a[k
17         ]);
18     printf("\n");
19
20     for (int k = 1; k < 10; k++)
21         if (ql_a[k-1] > ql_a[k]) { printf("FAIL\n"); return 1; }
22     printf("OK: array is sorted\n");
23     return 0;
24 }

```

QL 函数和全局符号以 ql_ 前缀导出，避免与 C 标准库的 qsort 产生名称冲突。

3.6.2 端到端运行脚本

run.sh 自动检测宿主平台 (macOS 用 cc, Linux/Kunpeng 用 gcc), 依次执行编译、汇编、链接和运行, 同时测试未优化与优化两个版本:

Listing 15: run.sh (关键步骤)

```

1  case "$(uname -s)" in
2      Darwin) TARGET=macos ; CC=cc ;;
3      *)      TARGET=linux ; CC=gcc ;;
4  esac
5  # 1. QL -> tokens/AST/QTAC/ARM
6  python3 src/qcc.py tests/qsort.ql -o build --target $TARGET
7
8  # 2. 未优化: 汇编 + 链接 + 运行
9  $CC -c build/qsort.s -o build/qsort_run.o

```

```

10 $CC -c runtime/main.c      -o build/main.o
11 $CC build/qsort_run.o build/main.o -o build/qsort_run
12 ./build/qsort_run
13
14 # 3. 优化后：汇编 + 链接 + 运行
15 $CC -c build/qsort.opt.s -o build/qsort_opt_run.o
16 $CC build/qsort_opt_run.o build/main.o -o build/qsort_opt_run
17 ./build/qsort_opt_run

```

4 实验结果

4.1 端到端运行结果

图 1 展示了在 Apple Silicon macOS (arm64, 即鲲鹏同架构) 上执行 `./run.sh` 的完整终端输出截图：

```

proj-homework-2 — rinalic@MacBook-Air-M4 — -zsh — 145x41
..oj-homework-2
[ ] ~/Local/Github/XJTU-Labs/Compiler/proj-homework-2 | on main ?8 | at 02:19:31 PM
./run.sh
host : Darwin arm64
target : macos (compiler: cc)

== Stage 1-4: QL -> Token -> AST -> QTAC -> ARM (+opt) ==
target = macos
compiled tests/qsort.ql -> build/qsort.*
  optimiser removed 18 instruction(s) (4 redundant branch, 14 redundant reload, 0 self-move)

== Assemble + link + run (unoptimised) ==
cc -c build/qsort.s -o build/qsort_run.o
cc -c runtime/main.c -o build/main.o
cc build/qsort_run.o build/main.o -o build/qsort_run
./build/qsort_run
before: 5 3 8 1 9 2 7 4 6 0
after : 0 1 2 3 4 5 6 7 8 9
OK: array is sorted

== Assemble + link + run (optimised) ==
cc -c build/qsort.opt.s -o build/qsort_opt_run.o
cc -c runtime/main.c -o build/main.o
cc build/qsort_opt_run.o build/main.o -o build/qsort_opt_run
./build/qsort_opt_run
before: 5 3 8 1 9 2 7 4 6 0
after : 0 1 2 3 4 5 6 7 8 9
OK: array is sorted

All artefacts are in build/:
main.o
qsort_opt_run
qsort_opt_run.o
qsort_run
qsort_run.o
qsort.ast
qsort.opt.s
qsort.qtac
qsort.s
qsort.sym
qsort.tokens

```

图 1: `run.sh` 完整运行结果 (macOS arm64)

截图中的关键信息逐行解读：

1. **宿主信息**：`host: Darwin arm64, target: macos (compiler: cc)`——系统自动选择了 macOS 目标模式和 Apple Clang 编译器。

2. **Stage 1-4 编译阶段**: `compiled tests/qsrt.q1 -> build/qsrt.*`, 说明词法、语法、QTAC、ARM 生成四个阶段均成功完成, 并同时报告优化器删除了 18 条指令 (4 条冗余跳转、14 条冗余加载、0 条自赋值)。
3. **未优化版本运行结果**:
 - before: 5 3 8 1 9 2 7 4 6 0——输入序列 (随机乱序)
 - after : 0 1 2 3 4 5 6 7 8 9——排序后的序列
 - OK: array is sorted——正确性检验通过
4. **优化版本运行结果**: 与未优化版本完全一致, 同样输出 OK: array is sorted, 证明优化不改变程序语义。
5. **生成的文件列表**: build/ 目录下包含 `qsrt.tokens`、`qsrt.ast`、`qsrt.qtac`、`qsrt.sym`、`qsrt.s`、`qsrt.opt.s` 等全部交付物, 以及汇编链接产生的目标文件和可执行文件。

4.2 各阶段输出统计

表 5: 各阶段输出文件统计

文件	内容	行数
<code>qsrt.tokens</code>	Token 序列	235
<code>qsrt.ast</code>	QAST 格式化输出	109
<code>qsrt.qtac</code>	QTAC 三地址码	83
<code>qsrt.sym</code>	符号表	11
<code>qsrt.s</code>	未优化 ARM64 汇编	241
<code>qsrt.opt.s</code>	优化后 ARM64 汇编	223

4.3 与参考 QTAC 对比分析

表 6: 本实现 QTAC 与材料参考 QTAC 的对比

方面	参考 QTAC	本实现 QTAC
全局声明	i80 a;	i80 a; (一致)
函数结构	define f(a,b){...}	一致
标签命名	l0, l1, ...	一致
临时变量命名	t0, t1, ...	一致
数组寻址	t = arr + idx (省略 $\times 8$)	t = idx * 8; t = arr + t (显式 $\times 8$)
条件跳转	IF q rop a THEN l ELSE l	一致
函数调用	PAR a; ...; q = CALL f, k	一致
顶层语句	直接列出 (不封装函数)	封装为 define entry(){...}

两者的结构性差异仅有两处：(1) 字节偏移的显式计算（对正确运行必要）；(2) 顶层语句的封装方式（使代码生成器只需处理函数，无需特殊处理顶层代码）。控制流、命名约定和调用协议均与参考完全一致。

5 总结

本实验成功实现了 QL 语言简易编译器 QCompiler 的完整路径 A (LL(1) 递归下降)，覆盖编译原理的全部核心阶段：

- **词法分析**：正确识别 QL 的 5 类 Token，处理行注释和排版字符规范化，对 qsort.ql 生成 235 个 Token。
- **语法分析与 AST 构建**：通过消除左递归、合并表达式/条件优先级梯级，实现了满足 LL(1) 条件的递归下降分析器，正确构建了完整的 QAST 树，所有结点类型均与 QASTNode 规范对应。
- **QTAC 中间代码生成**：生成符合规范的 QTAC 三地址码和符号表，控制流结构、函数调用协议与参考 QTAC 高度一致，数组字节偏移显式计算确保代码可直接运行。
- **目标代码生成**：按 Q2ARM 指令模板将 QTAC 映射为 AAPCS64 兼容的 ARM64 汇编，采用全溢出策略，无需手工分配寄存器；同时支持 Linux/Kunpeng (ADR) 和 macOS (ADRP/@PAGE) 两种全局地址化模式。
- **代码优化**：三遍窥孔优化删除 18 条冗余指令（约 7.5%），优化前后运行结果完全一致。
- **端到端验证**：在 Apple Silicon macOS (arm64，与鲲鹏同架构) 上一键运行，对 10 元素整型数组成功执行快速排序并通过正确性检验。

主要经验 本实验最具挑战性的环节是**文法变换**：QL 原始文法的二义性（ E 与 C 混用导致括号歧义）最终通过将两者合并为统一优先级梯级文法来解决，既消除了歧义，又避免了在 AST 中引入多余的条件结点。另一个关键细节是**数组地址的字节偏移**：参考 QTAC 省略了 $\times 8$ 的缩放，但在实际执行时必须显式计算，否则会访问错误的内存地址。

遗留问题 本实现采用全溢出策略，寄存器利用率较低；完整的图着色寄存器分配将是进一步提升生成代码质量的主要方向。此外，当前符号表对数组形参的类型记录较简单（仅记录 `int[]`），扩展为携带维度信息后可支持多维数组。

附录：实验源代码

A.1 词法分析器 (lexer.py)

```

1  import re
2
3  KEYWORDS = {"int", "void", "if", "else", "while", "return"}
4  MULTI_OPS = ["<=", ">=", "==", "!=", "&&", "||", "+="]
5  SINGLE_OPS = set("+-*/=<>!")
6  DELIMS = set("(){}[]; ,")
7  GLYPH_FIX = {"x": "*", "-": "-", "-": "-", "-": "-"}
8
9  class Token:
10     def __init__(self, type_, value, line, col):
11         self.type = type_; self.value = value
12         self.line = line; self.col = col
13     def __repr__(self): return f"({self.type}, {self.value})"
14
15  class LexError(Exception): pass
16
17  def normalise(text):
18     for bad, good in GLYPH_FIX.items(): text = text.replace(bad,
19                                                                good)
19     return text
20
21  def tokenize(text):
22     text = normalise(text)
23     tokens = []; line, col = 1, 1; i, n = 0, len(text)
24     def adv(k=1):
25         nonlocal i, col; i += k; col += k
26     while i < n:

```

```

27     c = text[i]
28     if c == "\n": line += 1; col = 1; i += 1; continue
29     if c in "\t\r": adv(); continue
30     if c == "/" and i+1 < n and text[i+1] == "/":
31         while i < n and text[i] != "\n": i += 1
32         continue
33     start_col = col
34     if c.isdigit():
35         m = re.match(r"[0-9]+", text[i:])
36         tokens.append(Token("NUM", m.group(0), line, start_col))
37         adv(len(m.group(0))); continue
38     if c.isalpha() or c == "_":
39         m = re.match(r"[a-zA-Z_][a-zA-Z0-9_]*", text[i:])
40         word = m.group(0)
41         ttype = "KEYWORD" if word in KEYWORDS else "ID"
42         tokens.append(Token(ttype, word, line, start_col))
43         adv(len(word)); continue
44     two = text[i:i+2]
45     if two in MULTI_OPS:
46         tokens.append(Token("OP", two, line, start_col)); adv(2)
47         ; continue
48     if c in SINGLE_OPS:
49         tokens.append(Token("OP", c, line, start_col)); adv();
50         continue
51     if c in DELIMS:
52         tokens.append(Token("DELIM", c, line, start_col)); adv()
53         ; continue
54     raise LexError(f"illegal character {c!r} at line {line}, col
55                     {col}")
56 tokens.append(Token("EOF", "$", line, col))
57 return tokens
58
59 def dump_tokens(tokens):
60     return "\n".join(f"({t.type}, {t.value})" for t in tokens if t.
61                     type != "EOF") + "\n"
62
63 if __name__ == "__main__":
64     import sys
65     sys.stdout.write(dump_tokens(tokenize(sys.stdin.read())))

```


A.2 LL(1) 递归下降分析器 (parser_ll1.py, 节选)

```
1 import qast
2
3 class ParseError(Exception): pass
4
5 class Parser:
6     def __init__(self, tokens): self.toks = tokens; self.pos = 0
7
8     @property
9     def cur(self): return self.toks[self.pos]
10    def at(self, value): return self.cur.value == value
11    def at_type(self, t): return self.cur.type == t
12    def advance(self): t = self.cur; self.pos += 1; return t
13    def expect(self, v):
14        if self.cur.value != v:
15            raise ParseError(f"line {self.cur.line}: expected {v!r}
16                               got {self.cur.value!r}")
17        return self.advance()
18
19    def parse_program(self):
20        self.expect("{"); decls, stmts = self.parse_decls_and_stmts()
21        self.expect("}"); return qast.Program(decls, stmts)
22
23    def parse_decls_and_stmts(self):
24        decls = []
25        while self.cur.value in ("int", "void"): decls.append(self.parse_decl())
26        stmts = []
27        while not self.at("}"): stmts.append(self.parse_stmt())
28        return decls, stmts
29
30    def parse_decl(self):
31        ret_type = self.advance().value
32        name = self.expect_type("ID").value
33        if self.at("("):
34            params = self.parse_params()
35            body = self.parse_block()
36            return qast.FuncDecl(name, ret_type, params, body)
37        dims = []
```

```

37         if self.at("["):
38             self.expect("["); dims.append(int(self.expect_type("NUM"
39                 ).value))
40             self.expect("]")
41             self.expect(";"); return qast.VarDecl(name, dims)
42
43     def _left_assoc(self, sub, ops):
44         node = sub()
45         while self.cur.value in ops:
46             op = self.advance().value; node = qast.Binary(op, node,
47                 sub())
48         return node
49
50     def parse_assign(self):
51         left = self.parse_logic_or()
52         if self.cur.value in ("=", "+="):
53             op = self.advance().value; right = self.parse_assign()
54             return qast.Binary(op, left, right)
55         return left
56
57     def parse_logic_or(self): return self._left_assoc(self.
58         parse_logic_and, ("||",))
59     def parse_logic_and(self): return self._left_assoc(self.
60         parse_equality, ("&&",))
61     def parse_equality(self): return self._left_assoc(self.
62         parse_relational, ("==", "!="))
63     def parse_relational(self): return self._left_assoc(self.
64         parse_additive,("<", "<=", ">", ">="))
65     def parse_additive(self): return self._left_assoc(self.parse_mul
66         , ("+", "-"))
67     def parse_mul(self): return self._left_assoc(self.parse_unary, (
68         "*", "/"))
69
70     def parse_postfix(self):
71         node = self.parse_primary()
72         while self.at("["):
73             self.expect("[")
74             if self.at("]"): self.expect("]") # arr[] 退化形式
75             else:
76                 idx = self.parse_expr(); self.expect("]")
77                 node = qast.Index(node, idx)

```

```

70         return node
71
72 def parse(tokens): return Parser(tokens).parse_program()

```

A.3 QTAC 生成器 (irgen.py, 节选)

```

1  WORD = 8
2
3  class IRGen:
4      def __init__(self):
5          self.code = []; self.tcount = 0; self.lcount = 0
6          self.globals = {}; self.scope = {}; self.symtab_lines = []
7
8      def new_temp(self): t = f"t{self.tcount}"; self.tcount += 1;
9          return t
10     def new_label(self): l = f"l{self.lcount}"; self.lcount += 1;
11         return l
12     def emit(self, s): self.code.append(s)
13
14     def gen_addr(self, e):
15         # 数组元素地址 = base + index * 8
16         base = self.gen_expr(e.base); idx = self.gen_expr(e.index)
17         idx = self._as_register(idx)
18         off = self.new_temp(); self.emit(f"{off} = {idx} * {WORD}")
19         addr = self.new_temp(); self.emit(f"{addr} = {base} + {off}")
20         return addr
21
22     def gen_cond(self, c, l_true, l_false):
23         if c.kind == "Binary" and c.op in self.REL:
24             a = self.gen_expr(c.left); b = self.gen_expr(c.right)
25             a = self._as_register(a)
26             self.emit(f"IF {a} {c.op} {b} THEN {l_true} ELSE {l_false}")
27         elif c.kind == "Binary" and c.op == "&&":
28             mid = self.new_label()
29             self.gen_cond(c.left, mid, l_false)
30             self.emit(f"LABEL {mid}")
31             self.gen_cond(c.right, l_true, l_false)
32         elif c.kind == "Unary" and c.op == "!":

```

```

31         self.gen_cond(c.operand, l_false, l_true) # 交换真假标
           签

```

A.4 窥孔优化器 (optimize.py)

```

1  import re
2
3  def optimize(asm_text):
4      lines = asm_text.splitlines()
5      stats = {"branches": 0, "reloads": 0, "selfmoves": 0, "removed":
6              0}
7
8      # 遍 1: 冗余跳转 (B Lx 紧接 Lx:)
9      out = []; i = 0
10     while i < len(lines):
11         m = re.match(r"^s*B\s+(\w+)$", lines[i])
12         if m and i+1 < len(lines) and lines[i+1].strip() == f"{m.
13             group(1)}:":
14             stats["branches"] += 1; i += 1; continue
15         out.append(lines[i]); i += 1
16     lines = out
17
18     # 遍 2: STR Xr,[sp,#k] 紧接 LDR Xr,[sp,#k] -> 删除 LDR
19     str_re = re.compile(r"^s*STR\s+(X\w+),\s*\[sp, #(\d+)\]$")
20     ldr_re = re.compile(r"^s*LDR\s+(X\w+),\s*\[sp, #(\d+)\]$")
21     out = []; i = 0
22     while i < len(lines):
23         s = str_re.match(lines[i])
24         l2 = ldr_re.match(lines[i+1]) if i+1 < len(lines) else None
25         if s and l2 and s.group(1)==l2.group(1) and s.group(2)==l2.
26             group(2):
27             out.append(lines[i]); stats["reloads"] += 1; i += 2;
28             continue
29         out.append(lines[i]); i += 1
30     lines = out
31
32     # 遍 3: MOV Xr, Xr -> 删除
33     out = []
34     for line in lines:
35         m = re.match(r"^s*MOV\s+(X\w+),\s*(X\w+)$", line)

```

```

32         if m and m.group(1) == m.group(2): stats["selfmoves"] += 1;
           continue
33     out.append(line)
34     stats["removed"] = stats["branches"]+stats["reloads"]+stats["
           selfmoves"]
35     return "\n".join(out)+"\n", stats

```

A.5 运行命令

一键端到端测试（自动检测平台）

```
./run.sh
```

手动运行各阶段（以 macOS 为例）

```

python3 src/lexer.py          < tests/qsort.q1          # 输出 Token 流
python3 src/parser_ll11.py    < tests/qsort.q1          # 输出 QAST
python3 src/irgen.py          < tests/qsort.q1          # 输出 QTAC
python3 src/codegen.py --macos < build/qsort.q1tac      # 输出 ARM64 (macOS)

```

完整编译（生成全部中间文件）

```
python3 src/qcc.py tests/qsort.q1 -o build --target macos
```

汇编 + 链接 + 运行 (macOS)

```

cc -c build/qsort.s -o build/qsort.o
cc -c runtime/main.c -o build/main.o
cc build/qsort.o build/main.o -o build/qsort_run
./build/qsort_run

```

鲲鹏/Linux 平台（--target linux 为默认值）

```

python3 src/qcc.py tests/qsort.q1 -o build
gcc -c build/qsort.s -o build/qsort.o
gcc -c runtime/main.c -o build/main.o
gcc build/qsort.o build/main.o -o build/qsort_run
./build/qsort_run

```